

Huffman Coding: Detailed Notes and C++ Implementation

Introduction:

Huffman Coding is a lossless data compression algorithm. It assigns variable-length codes to input characters, with shorter codes assigned to more frequent characters. It was developed by David A. Huffman in 1952.

How it works:

1. Frequency Table:

Count the frequency of each character in the input data.

2. Priority Queue (Min-Heap):

Build a min-heap with all characters as leaf nodes ordered by frequency.

3. Build Huffman Tree:

- Extract two nodes with the lowest frequency.
- Create a new node with these two nodes as children and frequency equal to their sum.
- Insert the new node back into the min-heap.
- Repeat until only one node (root) remains.

4. Generate Huffman Codes:

Traverse the tree:

- Left edge -> '0'
- Right edge -> '1'

Assign the resulting binary string to each character.

5. Encoding:

Replace each character in the input string with its Huffman code.

6. Decoding:

Use the Huffman tree to convert the binary string back to original characters.

C++ Implementation (explained):

```

#include <iostream>
#include <queue>
#include <unordered_map>
#include <vector>

using namespace std;

// A Huffman tree node
struct Node {
    char ch;
    int freq;
    Node *left, *right;
    Node(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
};

// Comparator for the min-heap
struct Compare {
    bool operator()(Node* l, Node* r) {
        return l->freq > r->freq;
    }
};

// Traverse the Huffman Tree and store codes in a map
void generateCodes(Node* root, string str, unordered_map<char, string>& huffmanCode) {
    if (!root) return;
    if (!root->left && !root->right) {
        huffmanCode[root->ch] = str;
    }
    generateCodes(root->left, str + "0", huffmanCode);
    generateCodes(root->right, str + "1", huffmanCode);
}

// Build Huffman Tree and encode text
void HuffmanCoding(string text) {
    unordered_map<char, int> freq;
    for (char ch : text) freq[ch]++;

    priority_queue<Node*, vector<Node*>, Compare> pq;
    for (auto pair : freq) {
        pq.push(new Node(pair.first, pair.second));
    }

    while (pq.size() != 1) {
        Node *left = pq.top(); pq.pop();

```

```

    Node *right = pq.top(); pq.pop();
    Node *sum = new Node(' ', left->freq + right->freq);
    sum->left = left;
    sum->right = right;
    pq.push(sum);
}

Node* root = pq.top();
unordered_map<char, string> huffmanCode;
generateCodes(root, "", huffmanCode);

string encoded = "";
for (char ch : text) encoded += huffmanCode[ch];

cout << "Huffman Codes:
";
for (auto pair : huffmanCode) {
    cout << pair.first << ": " << pair.second << '\n';
}

cout << "Encoded String: " << encoded << '\n';
}

```

Pros of Huffman Coding:

- Efficient compression for variable-length symbol frequencies.
- Always produces optimal prefix codes.
- Reduces average code length.

Cons of Huffman Coding:

- Needs to transmit the Huffman tree or table.
- Inefficient for small data sizes.
- Doesn't perform well with uniform frequency data.

Applications:

- File compression tools (e.g., ZIP, GZIP).
- Image compression (e.g., JPEG).
- Multimedia codecs (e.g., MP3, MPEG).
- Network data transmission optimization.

Conclusion:

Huffman Coding is one of the most fundamental and efficient algorithms for lossless data compression. Its simple concept and guaranteed optimality make it ideal for many real-world applications.